

Secuencias de parada y control de la generación

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 8 de 12 · Notas en español (es-ES)

La sesión 8 (19 de agosto de 2026) arrancó cerrando un control que el laboratorio del 17 de agosto ya había nombrado: las **secuencias de parada**. Pasáis una o más cadenas en la petición Messages. Cuando el modelo emite una cadena que coincide, la generación **se detiene**. El tokenizador sigue facturando los tokens realmente producidos — incluida la puntuación que subió el recuento a 14 en el demo 1,2,3,4.

Qué hacen — y qué no hacen — las secuencias de parada

Eslogan de Vijay: la función **termina la generación de contenido**; **no filtra** una respuesta más larga a posteriori. El modelo no escribe 1–100 por dentro y luego recorta a 5–10 para el cliente. Combinado con **prefill** (mensaje assistant que ya contiene el arranque de la respuesta), dirigís *dónde empieza y acaba la generación*, no un post-proceso de un ensayo oculto.

«SOLO CLAUDE» EN CLASE

El formador presentó las secuencias de parada como un control específico de Claude frente a «escribe un prompt que diga empieza en 5 y para en 10». Otros proveedores también tienen parámetros stop. Para el examen, conoced el campo `stop_sequences` y `stop_reason`; no montéis una guerra de vendors en el escrito. Confirmad el nombre vigente en la Messages API.

Prefill más stop: el ejemplo de contar

Mritunjay temía que el modelo «sepa» 1–100 y que los tokens de salida cuenten igual los números no usados. Corrección de Vijay: si el prefill hace que la continuación empiece en 5 y paráis en 10 (o en el token , 5 del ejemplo 1,2,3,4), el cliente solo recibe ese tramo. Los tokens de salida facturados son ese tramo, no un 1–100 secreto. Prefill + stop es un **control de prompting / decodificación**, no un segundo paso de LLM que trunca.

También limitó la técnica: brilla en contenido **secuencial / delimitado** (listas, código vallado, llaves JSON). Para razonamiento enmarañado o «haz todas las cuentas y dame una respuesta corta», las secuencias de parada son la herramienta equivocada — usad instrucciones más claras, interpolación o (luego) structured outputs / tools.

stop_reason que debéis reconocer

Lenguaje de clase	Significado típico	Qué hacer
Stop sequence	Apareció en la salida una cadena configurada	Parsead hasta (normalmente sin incluir) la cadena; es un corte controlado
	El modelo terminó de forma natural	

Lenguaje de clase	Significado típico	Qué hacer
EOS / fin de contenido / fin de turno		Tratadlo como completo salvo que el texto siga semánticamente inacabado
Max tokens / límite de tokens	Se alcanzó max_tokens	Reintentad con un techo mayor o diseñad un formato más corto

En el notebook, imprimir el Message entero mostraba `stop_reason` ligado a la secuencia ,5 tras emitir 1, 2, 3, 4. Quitad la secuencia de parada y veréis un final natural (o `max_tokens` si pedís del 1 al 10 000 con un techo diminuto).

Delimitadores que inventáis

Las paradas pueden ser vallas markdown (`` ` `), un marcador como `END`, incluso un nombre — siempre que la **misma** cadena esté en las instrucciones del prompt y en `stop_sequences`. Los agentes lo necesitan para que el volcado de un archivo no se alargue en charla. Si instruís «después del bloque JSON escribid el marcador `END`», la lista de stop debe incluir `END`.

Dónde encaja entre otros controles

- **max_tokens** — presupuesto duro; toscó.
- **stop_sequences** — halt en un marcador que elegisteis.
- **prefill** — empezar la generación a mitad de respuesta (assistant parcial).
- **temperature** — forma de la distribución, no una parada.
- **Structured outputs** (docs) — JSON acotado por esquema; en apps de 2026 preferid esto a trucos de valla cuando la API lo soporte para vuestro modelo.

Trampas de examen

- Stop ≠ filtro de contenido / clasificador de seguridad.
- El recuento de tokens de salida es la continuación generada, no una respuesta oculta completa.
- Inspeccionad siempre `stop_reason` antes de asumir que el JSON está cerrado.
- El streaming emite la misma parada; el último delta puede acabar en la secuencia.

DOCUMENTACIÓN FRENTE AL NOTEBOOK

Los nombres de parámetro (`stop_sequences`) y los valores de `stop_reason` deben coincidir con la **Messages API**, no con una captura borrosa.

Documentación oficial

- **Messages API**
<https://docs.claude.com/en/api/messages>
`stop_sequences`, `stop_reason`, `prefill` de assistant.
- **Visión general de prompting**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Cómo se dirige a Claude con roles y delimitadores.

- **Salidas estructuradas**

<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

Cuándo las restricciones de esquema ganan a los hacks JSON con stop.

- **Streaming**

<https://docs.claude.com/en/docs/build-with-claude/streaming>

Cómo se ven las paradas cuando los tokens llegan como deltas.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.